

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: CƠ SỞ DỮ LIỆU PHÂN TÁN

Giảng viên hướng dẫn : Kim Ngọc Bách
Nhóm học phần : 09
Nhóm bài tập lớn : 02

Các sinh viên thực hiện:

B22DCCN020	Lê Việt Anh
B22DCCN161	Hoàng Tùng Dương
B22DCCN743	Lâm Quốc Tú

Hà Nội – 2025

PHÂN CÔNG NHIỆM VỤ

STT	Công việc / Nhiệm vụ	SV Thực hiện	Thời hạn hoàn thành
1	Tổng hợp, viết báo cáo + hàm loadratings	Lê Việt Anh	08/06/2025
2	Viết hàm rangepartition + rangeinsert và các tài liệu liên quan	Lâm Quốc Tú	08/06/2025
3	Viết hàm roundrobinpartition + roundrobininsert và các tài liệu liên quan	Hoàng Tùng Dương	08/06/2025

NHÓM THỰC HIỆN TỰ ĐÁNH GIÁ

STT	Tên SV	Thái độ tham gia	Mức độ hoàn thành	Kỹ năng giao tiếp	Kỹ năng hợp tác
1	Lê Việt Anh	5	5	5	5
2	Lâm Quốc Tú	5	5	5	5
3	Hoàng Tùng Dương	5	5	5	5

Ghi chú:

- Thái độ tham gia: Đánh giá điểm thái độ tham gia công việc chung của nhóm (từ 0: không tham gia, đến 5: chủ động, tích cực).
- Mức hoàn thành CV: Đánh giá điểm mức độ hoàn thành công việc được giao (từ 0: không hoàn thành, đến 5: hoàn thành xuất sắc).
- Kỹ năng giao tiếp: Đánh giá điểm khả năng tương tác, giao tiếp trong nhóm (từ 0: không hoặc giao tiếp rất yếu, đến 5: giao tiếp xuất sắc).
- Kỹ năng hợp tác: Đánh giá điểm khả năng hợp tác, hỗ trợ lẫn nhau, giải quyết mâu thuẫn, xung đột

MỤC LỤC

PHÂN CÔNG NHIỆM VỤ	2
NHÓM THỰC HIỆN TỰ ĐÁNH GIÁ	3
MỤC LỤC	4
DANH MỤC HÌNH ẢNH	6
CHƯƠNG 1. GIỚI THIỆU BÀI TOÁN	7
1.1 Yêu cầu.....	7
1.2 Dữ liệu đầu vào.....	7
1.3 Các bước thực hiện.....	7
CHƯƠNG 2. KỸ THUẬT, CÔNG NGHỆ SỬ DỤNG	9
2.1 Ngôn ngữ lập trình: Python [1]	9
2.1.1 Giới thiệu chung về Python.....	9
2.1.2 Những lợi ích của Python.....	9
2.1.3 Ứng dụng của Python.....	9
2.1.3.1 Phát triển web server	9
2.1.3.2 Tự động hoá.....	9
2.1.3.3 Khoa học dữ liệu và học máy	10
2.1.3.4 Phát triển phần mềm	10
2.1.3.5 Kiểm thử chương trình	10
2.2 Hệ quản trị cơ sở dữ liệu: PostgreSQL [2].....	11
2.2.1 Giới thiệu chung về PostgreSQL.....	11
2.2.2 Tính năng nổi bật của PostgreSQL	11
2.2.3 Ưu điểm và nhược điểm của PostgreSQL	12
2.3 Thư viện hỗ trợ: Psycopg2 [3]	12
2.3.1 Giới thiệu chung về Psycopg2	12
2.3.2 Điểm nổi bật của Psycopg2	13
2.3.3 Ứng dụng của Psycopg2	13
CHƯƠNG 3. CHI TIẾT MÃ NGUỒN	15
3.1 Cơ sở lý thuyết	15
3.1.1 Phân mảnh ngang	15
3.1.2 Phân mảnh vòng tròn(Round-Robin Partitioning)	15
3.1.3 Phân mảnh theo khoảng (Range Partitioning).....	15
3.2 Phân tích chi tiết mã nguồn	16
3.2.1 Các hàm hỗ trợ.....	16

3.2.1.1 Hàm <code>getopenconnection()</code>	16
3.2.1.2 Hàm <code>create_db()</code>	17
3.2.1.3 Hàm <code>deleteAllPublicTables()</code>	17
3.2.2 Các hàm đọc dữ liệu và phân mảnh	19
3.2.2.1 Hàm <code>loadratings()</code>	19
3.2.2.2 Hàm <code>rangepartition()</code>	21
3.2.2.3 Hàm <code>rangeinsert()</code>	24
3.2.2.4 Hàm <code>roundrobinpartition()</code>	26
3.2.2.5 Hàm <code>roundrobininsert()</code>	28
CHƯƠNG 4. CÀI ĐẶT, KIỂM THỬ	29
4.1 Môi trường cài đặt	29
4.2 Kết quả triển khai	29
4.2.1 Đọc dữ liệu ratings	29
4.2.2 Phân mảnh theo khoảng	30
4.2.3 Chèn dữ liệu vào phân mảnh theo khoảng	31
4.2.4 Phân mảnh vòng tròn	32
4.2.5 Chèn dữ liệu vào phân mảnh vòng tròn	34
TÀI LIỆU THAM KHẢO	36

DANH MỤC HÌNH ẢNH

Hình 1 – Chi tiết hàm <code>getopenconnection()</code>	16
Hình 2 – Chi tiết hàm <code>create_db()</code>	17
Hình 3 – Chi tiết hàm <code>deleteAllPublicTables</code>	18
Hình 4 – Chi tiết hàm <code>loadratings()</code>	19
Hình 5 – Chi tiết hàm <code>rangepartition()</code>	21
Hình 6 – Chi tiết hàm <code>rangeinsert()</code>	24
Hình 7 – Chi tiết hàm <code>roundrobinpartition()</code>	26
Hình 8 – Chi tiết hàm <code>roundrobininsert()</code>	28
Hình 9 – Kết quả triển khai hàm <code>loadratings()</code>	29
Hình 10 – Dữ liệu trong bảng <code>ratings</code>	30
Hình 11 – Kết quả triển khai hàm <code>rangepartition()</code>	30
Hình 12 – Các bảng trong CSDL sau khi phân mảnh theo khoảng	31
Hình 13 – Dữ liệu trong bảng <code>range_part0</code>	31
Hình 14 – Kết quả triển khai hàm <code>rangeinsert()</code>	32
Hình 15 – Truy vấn kiểm tra dữ liệu trong bảng <code>range_part2</code>	32
Hình 16 – Kết quả triển khai hàm <code>roundrobinpartition()</code>	33
Hình 17 – Các bảng trong CSDL sau khi phân mảnh vòng tròn	33
Hình 18 – Dữ liệu trong bảng <code>rrobin_part0</code>	34
Hình 19 – Kết quả triển khai hàm <code>roundrobininsert()</code>	35
Hình 20 – Truy vấn kiểm tra dữ liệu trong bảng <code>rrobin_part0</code>	35
Hình 21 – Truy vấn kiểm tra dữ liệu trong bảng <code>rrobin_part1</code>	35

CHƯƠNG 1. GIỚI THIỆU BÀI TOÁN

1.1 Yêu cầu

Mô phỏng các phương pháp phân mảnh dữ liệu trên một hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở (ví dụ: PostgreSQL hoặc MySQL). Mỗi nhóm tạo một tập các hàm Python để tải dữ liệu đầu vào vào một bảng quan hệ, phân mảnh bảng này bằng các phương pháp phân mảnh ngang khác nhau, và chèn các bộ dữ liệu mới vào đúng phân mảnh.

1.2 Dữ liệu đầu vào

Dữ liệu đầu vào là một tập dữ liệu đánh giá phim được thu thập từ trang web MovieLens (<http://movielens.org>). Dữ liệu thô có trong tệp `ratings.dat`.

Tệp `ratings.dat` chứa 10 triệu đánh giá và 100.000 thẻ được áp dụng cho 10.000 bộ phim bởi 72.000 người dùng. Mỗi dòng trong tệp đại diện cho một đánh giá của một người dùng với một bộ phim, và có định dạng như sau:

```
UserID::MovieID::Rating::Timestamp
```

Các đánh giá được thực hiện trên thang điểm 5 sao, có thể chia nửa sao. Dấu thời gian (Timestamp) là số giây kể từ nửa đêm UTC ngày 1 tháng 1 năm 1970. Ví dụ nội dung tệp:

```
1::122::5::838985046
1::185::5::838983525
1::231::5::838983392
```

1.3 Các bước thực hiện

1. Tải tệp `rating.dat` từ trang MovieLens:
<http://files.grouplens.org/datasets/movielens/ml-10m.zip>
2. Cài đặt hàm Python `LoadRatings()` nhận vào một đường dẫn tuyệt đối đến tệp `rating.dat`.
`LoadRatings()` sẽ tải nội dung tệp vào một bảng trong PostgreSQL có tên `Ratings` với schema sau:
 - o `UserID (int)`
 - o `MovieID (int)`
 - o `Rating (float)`
3. Cài đặt hàm Python `Range_Partition()` nhận vào: (1) bảng `Ratings` trong PostgreSQL và (2) một số nguyên `N` là số phân mảnh cần tạo. `Range_Partition()` sẽ tạo `N` phân mảnh ngang của bảng `Ratings` và

lưu vào PostgreSQL. Thuật toán sẽ phân chia dựa trên N khoảng giá trị đồng đều của thuộc tính Rating.

4. Cài đặt hàm Python `RoundRobin_Partition()` nhận vào: (1) bảng Ratings trong PostgreSQL và (2) một số nguyên N là số phân mảnh cần tạo.

Hàm sẽ tạo N phân mảnh ngang của bảng Ratings và lưu chúng trong PostgreSQL, sử dụng phương pháp phân mảnh kiểu **vòng tròn (round robin)**.

5. Cài đặt hàm Python `RoundRobin_Insert()` nhận vào: (1) bảng Ratings trong PostgreSQL, (2) UserID, (3) ItemID, (4) Rating. `RoundRobin_Insert()` sẽ chèn một bộ mới vào bảng Ratings và vào đúng phân mảnh theo cách round robin.

6. Cài đặt hàm Python `Range_Insert()` nhận vào: (1) bảng Ratings trong PostgreSQL, (2) UserID, (3) ItemID, (4) Rating. `Range_Insert()` sẽ chèn một bộ mới vào bảng Ratings và vào đúng phân mảnh dựa trên giá trị của Rating.

CHƯƠNG 2. KỸ THUẬT, CÔNG NGHỆ SỬ DỤNG

2.1 Ngôn ngữ lập trình: Python [1]

2.1.1 Giới thiệu chung về Python

Python là một ngôn ngữ lập trình được sử dụng rộng rãi trong các ứng dụng web, phát triển phần mềm, khoa học dữ liệu và học máy. Các nhà phát triển sử dụng Python vì nó hiệu quả, dễ học và có thể chạy trên nhiều nền tảng khác nhau. Phần mềm Python được tải xuống miễn phí, tích hợp tốt với tất cả các loại hệ thống và giúp tăng tốc độ phát triển.

2.1.2 Những lợi ích của Python

- Cú pháp đơn giản, gần giống tiếng Anh giao tiếp
- Viết chương trình sử dụng ít dòng mã hơn các ngôn ngữ khác, giúp tăng tốc độ phát triển phần mềm
- Có lượng lớn thư viện hỗ trợ cho gần như mọi công việc
- Dễ dàng sử dụng kết hợp với các ngôn ngữ khác như Java, C và C++
- Cộng đồng khổng lồ, có thể tìm kiếm sự hỗ trợ dễ dàng khi gặp vấn đề
- Có thể dễ dàng tìm kiếm tài liệu, video, hướng dẫn về Python trên mạng
- Hỗ trợ những hệ điều hành phổ biến như Windows, macOS, Linux và Unix

2.1.3 Ứng dụng của Python

2.1.3.1 Phát triển web server

Web server bao gồm những hàm backend phức tạp mà các trang web thực hiện để hiển thị thông tin cho người dùng.

Python phù hợp với lập trình web server bởi vì ngôn ngữ này cung cấp nhiều thư viện bao gồm mã viết sẵn cho các hàm backend phức tạp. Các framework Python cung cấp tất cả những công cụ cần thiết để xây dựng ứng dụng web một cách nhanh chóng và dễ dàng.

2.1.3.2 Tự động hoá

Python phù hợp để viết các script tự động hoá như:

- Đổi tên một lượng lớn tệp cùng lúc
- Chuyển đổi tệp sang một loại tệp khác
- Loại bỏ các từ trùng trong tệp
- Thực hiện các phép toán cơ bản

- Gửi email
- Tải xuống nội dung
- Phân tích log cơ bản
- Tìm kiếm lỗi trong nhiều tệp

2.1.3.3 Khoa học dữ liệu và học máy

Khoa học dữ liệu là trích xuất thông tin quý giá từ dữ liệu, và học máy dạy máy tính tự động học hỏi từ dữ liệu và đưa ra các dự đoán chính xác. Các nhà khoa học dữ liệu sử dụng Python cho các tác vụ khoa học dữ liệu sau:

- Làm sạch dữ liệu
- Trích xuất và chọn lọc các đặc điểm đa dạng của dữ liệu
- Gán nhãn cho dữ liệu
- Tìm các số liệu thống kê khác nhau từ dữ liệu
- Trực quan hoá dữ liệu bằng cách sử dụng các biểu đồ và đồ thị

Các nhà khoa học dữ liệu sử dụng những thư viện học máy của Python để huấn luyện các mô hình học máy và xây dựng công cụ phân loại, giúp phân loại dữ liệu một cách chính xác. Các chuyên gia từ nhiều lĩnh vực lại sử dụng những công cụ phân loại này để phân loại hình ảnh, văn bản và lưu lượng truy cập mạng, nhận dạng giọng nói và nhận diện khuôn mặt. Các nhà khoa học cũng sử dụng Python cho học sâu, một dạng học máy nâng cao.

2.1.3.4 Phát triển phần mềm

Các nhà phát triển phần mềm thường sử dụng Python cho những tác vụ phát triển và ứng dụng phần mềm khác nhau, chẳng hạn như:

- Theo dõi lỗi trong mã nguồn
- Tự động xây dựng phần mềm
- Quản lý dự án phần mềm
- Phát triển nguyên mẫu phần mềm
- Phát triển các ứng dụng máy tính bằng cách sử dụng những thư viện GUI
- Phát triển từ các trò chơi điện tử

2.1.3.5 Kiểm thử chương trình

Kiểm thử phần mềm là quy trình kiểm tra xem kết quả thực tế từ phần mềm có khớp với kết quả mong đợi không để đảm bảo rằng phần mềm không có lỗi.

- Các nhà phát triển sử dụng unit test framework Python, chẳng hạn như Unittest, Robot và PyUnit, để kiểm thử các hàm do họ viết.
- Các kỹ sư kiểm thử phần mềm sử dụng Python để viết các trường hợp kiểm thử cho nhiều tình huống khác nhau.

2.2 Hệ quản trị cơ sở dữ liệu: PostgreSQL [2]

2.2.1 Giới thiệu chung về PostgreSQL

PostgreSQL là một hệ thống quản trị cơ sở dữ liệu quan hệ và đối tượng miễn phí và mã nguồn mở tiên tiến nhất hiện nay với khả năng mở rộng cao và tuân thủ các tiêu chuẩn kỹ thuật. Nó được thiết kế để xử lý một loạt các khối lượng công việc lớn, từ các máy tính cá nhân đến kho dữ liệu hoặc dịch vụ Web có nhiều người dùng đồng thời.

2.2.2 Tính năng nổi bật của PostgreSQL

- **Kiểu dữ liệu đa dạng:** PostgreSQL hỗ trợ nhiều kiểu dữ liệu như số nguyên, số thực, chuỗi, Boolean, ngày/giờ, mảng, phạm vi, UUID, JSON/JSONB, XML, key-value (Hstore), hình học và các kiểu dữ liệu tùy chỉnh.
- **Tính toàn vẹn dữ liệu:** PostgreSQL cung cấp các cơ chế để đảm bảo tính toàn vẹn dữ liệu như ràng buộc UNIQUE, NOT NULL, Primary Keys, Foreign Keys và ràng buộc loại trừ. Ngoài ra, nó còn hỗ trợ khóa hàm số/explicit locks và khóa khuyến nghị/advisory locks.
- **Đồng quy và hiệu suất:** PostgreSQL cung cấp các phương pháp lập danh mục (indexing) như B-tree, multicolumn, expressions, partial và các phương pháp lập danh mục nâng cao như GiST, SP-Gist, KNN Gist, GIN, BRIN, Bloom filters. Nó cũng có trình lập kế hoạch/trình tối ưu hóa truy vấn phức tạp, hỗ trợ quét chỉ trên danh mục (index-only scan) và thống kê đa cột.
- **Giao tác và đồng thời:** PostgreSQL hỗ trợ giao tác ACID (Atomicity, Consistency, Isolation, Durability) và giao tác dạng nest thông qua lưu điểm (savepoints). Nó cũng hỗ trợ điều khiển đồng thời nhiều phiên bản (MVCC) và truy vấn đọc song song.
- **Phân vùng bảng:** PostgreSQL cho phép phân vùng bảng để tăng hiệu suất truy vấn và quản lý dữ liệu lớn.
- **Độ tin cậy và phục hồi:** PostgreSQL sử dụng ghi nhật ký ghi trước (Write-ahead Logging - WAL) để đảm bảo độ tin cậy và hỗ trợ khôi phục điểm-thời gian (Point-in-time Recovery - PITR) và active standbys.
- **Bảo mật:** PostgreSQL cung cấp các phương pháp xác thực như GSSAPI, SSPI, LDAP, SCRAM-SHA-256, Certificate và hệ thống kiểm soát truy cập mạnh mẽ. Nó cũng hỗ trợ bảo mật cấp độ cột và hàng.

- **Khả năng mở rộng:** PostgreSQL có khả năng mở rộng thông qua các phương pháp lưu trữ và hỗ trợ nhiều ngôn ngữ thủ tục như PL/PGSQL, Perl, Python và nhiều ngôn ngữ khác. Ngoài ra, nó hỗ trợ tích hợp với các cơ sở dữ liệu và ứng dụng khác thông qua giao diện SQL chuẩn và cung cấp nhiều tiện ích mở rộng bổ sung như PostGIS.
- **Tìm kiếm văn bản:** PostgreSQL hỗ trợ các bộ ký tự quốc tế thông qua ICU collations và cung cấp khả năng tìm kiếm văn bản toàn diện.

2.2.3 Ưu điểm và nhược điểm của PostgreSQL

Ưu điểm:

- Dễ sử dụng
- Hỗ trợ cho ứng dụng web động
- Khả năng lưu trữ nhật ký và khôi phục
- Mã nguồn mở miễn phí
- Đối tượng hoá dữ liệu
- Cộng đồng hỗ trợ đông đảo
- Giảm thiểu công việc bảo trì

Nhược điểm:

- Quản lý không thuộc về một tổ chức cụ thể
- Tập trung vào khả năng tương thích
- Hỗ trợ ứng dụng nguồn mở hạn chế
- Hiệu suất chậm hơn so với MySQL

2.3 Thư viện hỗ trợ: Psycopg2 [3]

2.3.1 Giới thiệu chung về Psycopg2

Psycopg2 là một bộ chuyển đổi phổ biến nhờ bộ tính năng toàn diện, độ ổn định cao và khả năng mở rộng, qua đó đã trở thành lựa chọn yêu thích của các nhà phát triển Python khi làm việc với PostgreSQL. Psycopg2 được triển khai bằng các phần mở rộng viết bằng ngôn ngữ C, điều này góp phần nâng cao hiệu suất hoạt động.

Psycopg2 hỗ trợ nhiều tính năng của PostgreSQL, bao gồm con trỏ phía máy chủ (server-side cursors), thông báo bất đồng bộ (asynchronous notifications) và các lệnh COPY. Ngoài ra, nó còn an toàn trong môi trường đa luồng và có khả năng quản lý kết nối. Psycopg2 được sử dụng rộng rãi là nhờ vào độ tin cậy và khả năng tương thích với nhiều phiên bản của PostgreSQL và Python, trở thành một lựa chọn linh hoạt cho nhiều loại ứng dụng khác nhau.

2.3.2 Điểm nổi bật của Psycpg2

- **Quản lý kết nối:** Psycpg2 nổi bật trong việc thiết lập và duy trì các kết nối vững chắc giữa các ứng dụng Python và cơ sở dữ liệu PostgreSQL. Nó đảm bảo một kênh truyền dữ liệu ổn định và không bị gián đoạn, từ đó nâng cao hiệu quả trong việc trao đổi và truyền thông tin dữ liệu.
- **Thực thi truy vấn SQL hiệu quả:** Psycpg2 có khả năng thực thi truy vấn SQL một cách chính xác và nhanh chóng. Nó xử lý tốt nhiều tác vụ khác nhau như truy xuất dữ liệu, chỉnh sửa bản ghi và thực hiện các thao tác phức tạp, đảm bảo hiệu suất tối ưu và độ chính xác cao.
- **Đồng bộ dữ liệu theo thời gian thực:** Psycpg2 hỗ trợ phát triển các ứng dụng theo thời gian thực bằng cách đảm bảo mã Python luôn được đồng bộ hóa liên tục với cơ sở dữ liệu PostgreSQL. Tính năng này giúp xây dựng các ứng dụng phản hồi nhanh, dựa trên dữ liệu, có khả năng thích ứng hiệu quả với những thay đổi dữ liệu liên tục và linh hoạt.
- **Tính vững chắc:** Psycpg2 được công nhận về độ ổn định và độ tin cậy, khiến nó trở thành lựa chọn lý tưởng cho các ứng dụng quan trọng, đòi hỏi độ chính xác cao. Thư viện này có khả năng xử lý nhiều tính năng của PostgreSQL, các kiểu dữ liệu phức tạp và đối tượng lớn, đảm bảo độ chính xác và độ tin cậy cần thiết cho các dự án có yêu cầu cao.
- **Tương thích chéo:** Psycpg2 tích hợp mượt mà với nhiều phiên bản khác nhau của Python và PostgreSQL, mang lại khả năng tương thích chéo và tính linh hoạt cho các dự án. Điều này đảm bảo rằng mã Python vẫn hoạt động ổn định trên nhiều môi trường khác nhau.
- **Cộng đồng sôi nổi:** Psycpg2 được hưởng lợi từ quá trình phát triển liên tục và một cộng đồng hỗ trợ nhiệt tình. Sự cam kết cải tiến này giúp Psycpg2 luôn được cập nhật với những tiến bộ mới nhất của Python và PostgreSQL, khiến nó trở thành một lựa chọn đáng tin cậy cho việc tương tác giữa Python và PostgreSQL.
- **Tăng cường bảo mật:** Psycpg2 ưu tiên bảo mật dữ liệu bằng cách cung cấp các tính năng mạnh mẽ, bao gồm hỗ trợ kết nối SSL. Lớp bảo mật bổ sung này giúp bảo vệ dữ liệu nhạy cảm, duy trì tính toàn vẹn và bảo mật thông tin trong các giao dịch.

2.3.3 Ứng dụng của Psycpg2

- Phân tích và thống kê dữ liệu
- Phát triển web

- Ứng dụng cho doanh nghiệp
- Các ứng dụng IoT và thời gian thực
- Nghiên cứu khoa học

CHƯƠNG 3. CHI TIẾT MÃ NGUỒN

3.1 Cơ sở lý thuyết

3.1.1 Phân mảnh ngang

Phân mảnh ngang là một kỹ thuật phân chia cơ sở dữ liệu, trong đó một quan hệ **R** được chia thành nhiều phần nhỏ, mỗi phần chứa một tập hợp các bộ dữ liệu thoả mãn một điều kiện nhất định.

Phân mảnh ngang giúp tối ưu truy vấn, tăng tính sẵn sàng dữ liệu và khả năng phân phối dữ liệu theo vùng, giảm chi phí truyền thông trong hệ thống phân tán.

3.1.2 Phân mảnh vòng tròn(Round-Robin Partitioning)

Phân mảnh vòng tròn là một phương pháp phân mảnh ngang trong cơ sở dữ liệu, được sử dụng để phân chia các bản ghi dữ liệu của một bảng thành nhiều phần theo một cách tuần tự và xoay vòng, không quan tâm đến nội dung dữ liệu.

Phân mảnh vòng tròn sử dụng một nguyên tắc đơn giản: các bản ghi được phân bổ lần lượt vào các phân vùng theo thứ tự, và sau khi hết một vòng, quá trình phân bổ bắt đầu lại từ đầu, giống như kim đồng hồ quay vòng.

Ưu điểm:

- Đơn giản trong triển khai
- Phân phối dữ liệu đồng đều
- Hiệu quả trong xử lý song song

Nhược điểm:

- Không tối ưu cho truy vấn có điều kiện
- Không tận dụng được tính chất dữ liệu

3.1.3 Phân mảnh theo khoảng (Range Partitioning)

Phân mảnh theo khoảng là kỹ thuật phân mảnh ngang trong đó các bản ghi của một bảng được chia thành nhiều phân vùng dựa trên phạm vi giá trị của một hoặc nhiều cột. Mỗi phân vùng tương ứng với một khoảng giá trị xác định trước.

Phân vùng theo khoảng đặc biệt hiệu quả trong các trường hợp dữ liệu có đặc điểm tuyến tính hoặc thời gian — như ngày tháng, ID tăng dần, hoặc các thuộc tính định lượng.

Nguyên tắc của phân mảnh theo khoảng: khi một bản ghi mới được chèn vào, hệ thống sẽ so sánh giá trị khóa phân mảnh của bản ghi với các giá trị biên mà người dùng đã định nghĩa để xác định bản ghi này thuộc phân vùng nào. Việc này đảm bảo rằng các

bản ghi có giá trị gần nhau về mặt dữ liệu sẽ được lưu cùng một nơi, giúp tăng hiệu quả truy vấn theo phạm vi.

Ưu điểm:

- Tối ưu cho truy vấn có điều kiện
- Tăng khả năng mở rộng

Nhược điểm:

- Phân phối dữ liệu không đều
- Khó duy trì nếu dữ liệu tăng trưởng động
- Cần xác định rõ các giá trị biên từ đầu

3.2 Phân tích chi tiết mã nguồn

Trong phần này, nhóm chúng em sẽ phân tích chi tiết các hàm có trong file Interface.py

3.2.1 Các hàm hỗ trợ

3.2.1.1 Hàm *getopenconnection()*

```
# Tạo kết nối đến db
def getopenconnection(user='postgres', password='1234', dbname='postgres', host='localhost'):
    conn = psycopg2.connect(
        dbname=dbname,
        user=user,
        password=password,
        host=host
    )
    print(f"Connected to {dbname}")
    return conn
```

Hình 1 – Chi tiết hàm *getopenconnection()*

Mục đích: Tạo kết nối đến cơ sở dữ liệu PostgreSQL nhờ phương thức connect() của thư viện psycopg2, cho phép ta tương tác với cơ sở dữ liệu từ chương trình Python.

Các tham số:

- user: Tên người dùng của cơ sở dữ liệu
- password: Mật khẩu để đăng nhập cơ sở dữ liệu
- dbname: Tên cơ sở dữ liệu muốn kết nối
- host: Địa chỉ của máy chủ cơ sở dữ liệu

Phân tích chi tiết:

- Được sử dụng để tạo kết nối đến một cơ sở dữ liệu trong PostgreSQL.
- Hàm sẽ trả về đối tượng kết nối đến cơ sở dữ liệu đó.

- Nếu không truyền tham số gì, hàm sẽ tạo kết nối đến cơ sở dữ liệu mặc định của PostgreSQL.

3.2.1.2 Hàm *create_db()*

```
# Tạo db mới
def create_db(dbname):
    # Tạo kết nối đến db mặc định
    conn = getopenconnection()

    # Thiết lập auto commit
    conn.set_isolation_level(psycopg2.extensions.ISOLATION_LEVEL_AUTOCOMMIT)

    # Tạo một con trỏ để thực hiện truy vấn
    cur = conn.cursor()

    # Kiểm tra xem đã tồn tại db cần tạo hay chưa
    cur.execute("SELECT COUNT(*) FROM pg_catalog.pg_database WHERE datname = %s", (dbname,))
    count = cur.fetchone()[0]

    if count == 0:
        # Tạo db mới
        createQuery = sql.SQL("CREATE DATABASE {dbname}").format(
            dbname=sql.Identifier(dbname)
        )
        cur.execute(createQuery)
        print(f"Created database {dbname}")
    else:
        print(f"Database {dbname} already exists")

    # Đóng kết nối
    cur.close()
    conn.close()
```

Hình 2 – Chi tiết hàm *create_db()*

Mục đích: Tạo mới một cơ sở dữ liệu nếu nó chưa tồn tại.

Các tham số:

- dbname: Tên cơ sở dữ liệu muốn tạo

Phân tích chi tiết:

- Tạo kết nối đến cơ sở dữ liệu mặc định.
- Thiết lập auto commit để tự động lưu những thay đổi đối với cơ sở dữ liệu.
- Tạo một con trỏ để thực hiện truy vấn.
- Kiểm tra xem cơ sở dữ liệu cần tạo đã tồn tại hay chưa.
- Nếu đã tồn tại, sẽ in ra thông báo cho người dùng.
- Nếu chưa, tạo cơ sở dữ liệu mới với tên là tham số truyền vào.
- Đóng con trỏ và đóng kết nối đến cơ sở dữ liệu mặc định sau khi hoàn thành.

3.2.1.3 Hàm *deleteAllPublicTables()*

```
# Xoá tất cả bảng public của db
def deleteAllPublicTables(openconnection):
    cur = openconnection.cursor()

    cur.execute("SELECT table_name FROM information_schema.tables WHERE table_schema = 'public'")
    l = []
    for row in cur:
        l.append(row[0])
    for tablename in l:
        cur.execute("drop table if exists {0} CASCADE".format(tablename))

    cur.close()
    openconnection.commit()
```

Hình 3 – Chi tiết hàm deleteAllPublicTables

Mục đích: Xoá toàn bộ bảng trong schema public của một cơ sở dữ liệu, được sử dụng trong quá trình kiểm thử để reset toàn bộ dữ liệu, tránh gặp lỗi khi chạy test.

Các tham số:

- openconnection: đối tượng kết nối đến cơ sở dữ liệu muốn reset

Phân tích chi tiết:

- Tạo một con trỏ để thực hiện truy vấn.
- Lấy danh sách tất cả các bảng có trong schema public.
- Duyệt qua danh sách này và xoá từng bảng nếu chúng tồn tại.
- Sử dụng CASCADE để đảm bảo xoá luôn các phụ thuộc như khoá ngoại, ...
- Cuối cùng đóng con trỏ và commit để lưu thay đổi.

3.2.2 Các hàm đọc dữ liệu và phân mảnh

3.2.2.1 Hàm loadratings()

```
def loadratings(ratingstablename, ratingsfilepath, openconnection):
    cur = openconnection.cursor()

    # Tạo bảng ratings
    # Sử dụng sql.Identifier để tránh tấn công SQL Injection
    createQuery = sql.SQL("""
        CREATE TABLE IF NOT EXISTS {table} (
            UserId INT,
            extra1 CHAR,
            MovieId INT,
            extra2 CHAR,
            Rating FLOAT,
            extra3 CHAR,
            timestamp BIGINT
        );
    """).format(table=sql.Identifier(ratingstablename))
    cur.execute(createQuery)
    print(f"Created table {ratingstablename}")

    # Đọc dữ liệu từ file và copy vào bảng
    with open(ratingsfilepath, 'r') as f:
        cur.copy_from(f, ratingstablename, sep=':')

    # Xóa các cột thừa
    alterQuery = sql.SQL("""
        ALTER TABLE {table}
        DROP COLUMN extra1,
        DROP COLUMN extra2,
        DROP COLUMN extra3,
        DROP COLUMN timestamp;
    """).format(table=sql.Identifier(ratingstablename))
    cur.execute(alterQuery)
    print(f"Data inserted into {ratingstablename} successfully")

    cur.close()
    openconnection.commit()
```

Hình 4 – Chi tiết hàm loadratings()

Mục đích: Tạo bảng ratings với dữ liệu được lấy từ file ratings.dat bằng phương thức copy của psycopg2

Các tham số:

- ratingstablename: tên bảng ratings
- ratingsfilepath: đường dẫn đến file ratings.dat
- openconnection: đối tượng kết nối đến cơ sở dữ liệu mà ta muốn tạo bảng ratings

Phân tích chi tiết:

- Tạo một con trỏ để thực hiện truy vấn.
- Tạo bảng ratings với các trường: UserId, extra1, MovieId, extra2, Rating, extra3, timestamp.
- Ta chọn cách đọc dữ liệu bằng phương thức copy_from() thay vì INSERT hay BATCH INSERT để tối ưu tốc độ truy vấn.
- Đọc dữ liệu từ file ratings.dat và lưu vào bảng ratings bằng phương thức copy_from(). Ở đây ta thiết lập ký tự ngăn cách là “:” mà không phải “::” vì phương thức copy_from() giới hạn một ký tự ngăn cách duy nhất.
- Điều này khiến cho dữ liệu trong bảng ratings có dạng:
 UserId – Khoảng trống – MovieId – Khoảng trống – Rating – Khoảng trống – timestamp
- Vì vậy, ta cần xoá những cột trống và cột timestamp không cần sử dụng.
- Thực hiện truy vấn ALTER TABLE để xoá các cột extra1, extra2, extra3 và timestamp.
- Cuối cùng đóng con trỏ và commit để lưu thay đổi.

3.2.2.2 Hàm *rangepartition()*

```
def rangepartition(ratingtablename, numberofpartitions, openconnection):
    # Số phân vùng phải là số nguyên dương lớn hơn 0
    if numberofpartitions <= 0:
        return

    try:
        connection = openconnection
        cursor = connection.cursor()

        RANGE_TABLE_PREFIX = 'range_part'

        # Xóa các bảng phân vùng cũ (nếu có)
        cursor.execute("""
            SELECT tablename
            FROM pg_tables
            WHERE tablename LIKE 'range_part%';
        """)
        old_tables = cursor.fetchall()
        for table_tuple in old_tables:
            table_name = table_tuple[0]
            cursor.execute(f"DROP TABLE IF EXISTS {table_name};")

        # Bảng metadata được tạo để lưu thông tin phân vùng
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS metadata (
                partition_type TEXT,
                num_partitions INTEGER,
                range_boundaries TEXT
            );
        """)

        cursor.execute("DELETE FROM metadata WHERE partition_type = 'range';")

        # Tính các biên phân vùng
        max_rating = 5.0
        min_rating = 0.0
        delta = (max_rating - min_rating) / numberofpartitions
        boundaries = [min_rating + i * delta for i in range(numberofpartitions + 1)]

        # Làm tròn các giá trị biên
        boundaries = [round(b * 100 + 0.5) / 100 for b in boundaries]
        boundaries_str = ",".join(str(b) for b in boundaries)

        # Lưu thông tin phân vùng vào bảng metadata
        cursor.execute("""
            INSERT INTO metadata (partition_type, num_partitions, range_boundaries)
            VALUES (%s, %s, %s);
        """, ('range', numberofpartitions, boundaries_str))

        # Tạo và phân phối dữ liệu vào các bảng phân vùng
        for i in range(numberofpartitions):
            table_name = f"{RANGE_TABLE_PREFIX}{i}"

            # Tạo bảng phân vùng nếu chưa tồn tại
            cursor.execute(f"""
                CREATE TABLE IF NOT EXISTS {table_name} (
                    userid INTEGER,
                    movieid INTEGER,
                    rating FLOAT
                );
            """)

            # Xóa dữ liệu cũ trong bảng phân vùng (nếu có)
            cursor.execute(f"DELETE FROM {table_name};")

            # Phân phối dữ liệu cho các phân vùng:
            # Phân vùng đầu tiên
            if i == 0:
                cursor.execute(f"""
                    INSERT INTO {table_name} (userid, movieid, rating)
                    SELECT userid, movieid, rating
                    FROM {ratingtablename}
                    WHERE rating >= %s AND rating <= %s;
                """, (boundaries[i], boundaries[i + 1]))
            # Các phân vùng còn lại:
            else:
                cursor.execute(f"""
                    INSERT INTO {table_name} (userid, movieid, rating)
                    SELECT userid, movieid, rating
                    FROM {ratingtablename}
                    WHERE rating > %s AND rating <= %s;
                """, (boundaries[i], boundaries[i + 1]))

        openconnection.commit()
        cursor.close()
    except Exception as e:
        openconnection.rollback()
        print("Range Partitioning failed")
        print(e)
        raise e
```

Hình 5 – Chi tiết hàm *rangepartition()*

Mục đích: Tạo N phân mảnh ngang theo khoảng của bảng ratings và lưu vào PostgreSQL. Thuật toán sẽ phân chia dựa trên N khoảng giá trị đồng đều của thuộc tính Rating, mỗi bản ghi trong bảng ratings sẽ được phân vào một vùng cụ thể dựa trên giá trị Rating của nó.

Các tham số:

- ratingtablename: là bảng ratings cần phân mảnh
- numberofpartitions: một số nguyên N là số phân vùng cần tạo
- openconnection: đối tượng kết nối đến cơ sở dữ liệu

Phân tích chi tiết:

- Hàm chỉ hoạt động nếu người dùng nhập số phân mảnh lớn hơn 0.
- Khởi tạo một con trỏ để thực hiện truy vấn thông qua đối tượng kết nối openconnection.
- Hằng số RANGE_TABLE_PREFIX được sử dụng để làm tiền tố cho tên các bảng phân mảnh. Các bản phân mảnh sẽ có tên dạng range_part0, range_part1, ..., giúp phân biệt một cách dễ dàng.
- Tìm các bảng phân vùng cũ, duyệt qua từng bảng và sử dụng câu lệnh DROP TABLE IF EXISTS để xóa các bảng này nếu chúng tồn tại. Điều này đảm bảo xóa hết các phân mảnh cũ, tránh gây lỗi khi tạo phân mảnh mới.
- Tạo bảng metadata gồm các thuộc tính: partition_type: Loại phân mảnh, num_partitions: Số lượng phân mảnh, range_boundaries: Danh sách các giá trị biên của phân mảnh.
- Chia khoảng giá trị của Rating thành N phân mảnh, mỗi phân mảnh sẽ có một khoảng giá trị đồng đều. Tính độ dài của mỗi khoảng theo công thức:
$$\Delta = (max_rating - min_rating) / N$$
Với max_rating = 5.0, min_rating = 0.0 và N là số phân mảnh cần tạo.
- Boundaries là danh sách các biên của phân vùng. Vòng lặp sẽ tạo ra N + 1 biên (do biên đầu tiên là min_rating và biên cuối cùng là max_rating). Các biên này là các điểm cắt giữa các phân mảnh, giúp xác định rõ ràng mỗi phân mảnh sẽ chứa những bản ghi có giá trị Rating trong khoảng nào.
- Các giá trị biên được làm tròn đến 2 chữ số thập phân, sau đó được nối với nhau, ngăn cách bằng dấu “,”. Sau đó lưu các thông tin phân vùng vào bảng metadata.
- Tạo các bảng phân mảnh (range_part0, range_part1, ...). Các bảng là phân mảnh ngang của bảng ratings nên có cấu trúc gồm 3 cột UserId, MovieId, Rating giống bảng ratings.

- Trước khi chèn dữ liệu mới vào bảng phân mảnh, xóa toàn bộ dữ liệu cũ trong các bảng phân mảnh đó để tránh trùng lặp dữ liệu từ các bảng trong lần chạy trước, nếu có.
- Phân phối dữ liệu vào các phân mảnh:
 - Với phân mảnh đầu tiên ($i == 0$): Dữ liệu từ bảng Ratings được chèn vào bảng phân mảnh đầu tiên (range_part0) nếu giá trị rating của bản ghi nằm trong khoảng boundaries[0] và boundaries[1].
 - Với các phân mảnh còn lại: câu lệnh INSERT INTO sẽ chèn dữ liệu từ bảng Ratings vào các bảng phân mảnh tiếp theo (range_part1, range_part2, ...) với các điều kiện tương ứng.
- Cuối cùng commit để lưu tất cả thay đổi đã thực hiện và đóng con trỏ.

3.2.2.3 Hàm *rangeinsert()*

```
def rangeinsert(ratingtablename, userid, itemid, rating, openconnection):
    # Kiểm tra điều kiện rating hợp lệ
    if not (0 ≤ rating ≤ 5):
        raise Exception("Rating must be between 0 and 5.")

    # Kiểm tra UserID và MovieID hợp lệ
    if not (isinstance(userid, int) and isinstance(itemid, int) and userid > 0 and itemid > 0):
        raise Exception("UserID and MovieID must be positive integers.")

    try:
        connection = openconnection
        cursor = connection.cursor()

        RANGE_TABLE_PREFIX = 'range_part'

        # Kiểm tra nếu dữ liệu đã tồn tại trong bảng gốc
        cursor.execute(f"""
            SELECT 1 FROM {ratingtablename}
            WHERE userid = %s AND movieid = %s AND rating = %s;
            """, (userid, itemid, rating))

        if cursor.fetchone():
            print("Dữ liệu đã tồn tại trong Ratings")
            return

        # Chèn dữ liệu vào bảng chính
        cursor.execute(f"""
            INSERT INTO {ratingtablename} (userid, movieid, rating)
            VALUES (%s, %s, %s);
            """, (userid, itemid, rating))

        # Lấy thông tin phân vùng từ metadata
        cursor.execute("SELECT num_partitions FROM metadata WHERE partition_type = 'range';")
        num_partitions = cursor.fetchone()[0]

        # Tính chỉ số phân vùng dựa trên rating
        partition_size = round(5.0 / num_partitions, 2)
        partition_index = int(rating / partition_size)
        if rating % partition_size == 0 and rating != 0:
            partition_index -= 1

        # Chèn vào phân mảnh tương ứng
        table_name = f"{RANGE_TABLE_PREFIX}{partition_index}"
        cursor.execute(f"""
            INSERT INTO {table_name} (userid, movieid, rating)
            VALUES (%s, %s, %s);
            """, (userid, itemid, rating))

        openconnection.commit()
        cursor.close()
    except Exception as e:
        openconnection.rollback()
        print("Insert Range failed")
        print(e)
        raise e
```

Hình 6 – Chi tiết hàm *rangeinsert()*

Mục đích: Chèn một bản ghi vào bảng ratings, sau đó lấy thông tin bảng phân vùng tương ứng để phân phối bản ghi này dựa trên giá trị Rating.

Các tham số:

- ratingtablename: là bảng ratings trong PostgreSQL
- userid, itemid, rating: dữ liệu của một bản ghi
- openconnection: đối tượng kết nối đến cơ sở dữ liệu

Phân tích chi tiết:

- Kiểm tra tính hợp lệ của bản ghi trước khi chèn vào bảng:
 - userid và itemid phải là số nguyên dương.
 - rating phải nằm trong khoảng $[0, 5]$.
 - Bản ghi này phải chưa tồn tại trong cơ sở dữ liệu
 - Nếu bản ghi không hợp lệ sẽ thông báo lỗi cho người dùng và dừng hàm.
- Nếu bản ghi hợp lệ, ta chèn bản ghi vào bảng rating bằng lệnh INSERT
- Lấy thông tin phân vùng từ metadata sau đó tính chỉ số phân vùng dựa trên giá trị Rating
- Tính độ dài mỗi phân vùng và làm tròn kết quả đến 2 chữ số thập phân:
$$partition_size = round(5.0/num_partitions, 2)$$
- Tính chỉ số phân vùng:
$$partition_index = int(rating/partition_size)$$
- Điều chỉnh chỉ số phân vùng sao cho nó không bị chèn vào phân vùng sau nếu giá trị rating là một biên phân vùng chính xác:
- Chèn bản ghi vào phân vùng đã tính được ở trên
- Cuối cùng commit để lưu tất cả thay đổi đã thực hiện và đóng con trỏ.

3.2.2.4 Hàm *roundrobinpartition()*

```
def roundrobinpartition(ratingstablename, numberofpartitions, openconnection):
    """
    Chia dữ liệu bảng rating thành nhiều partition theo round robin.
    """
    with openconnection.cursor() as cur:
        # Xóa các bảng partition cũ (nếu có)
        cur.execute(sql.SQL("""
            SELECT tablename
            FROM   pg_tables
            WHERE  schemaname = 'public'
            AND    tablename LIKE %s
            """), (RROBIN_TABLE_PREFIX + '%',))
        old_parts = [t[0] for t in cur.fetchall()]
        for t in old_parts:
            cur.execute(sql.SQL("DROP TABLE IF EXISTS {} CASCADE").format(sql.Identifier(t)))
        # Tạo các bảng partition mới
        for i in range(numberofpartitions):
            cur.execute(sql.SQL("""
                CREATE UNLOGGED TABLE {part} (
                    userid   INTEGER,
                    movieid  INTEGER,
                    rating    FLOAT
                );
            """).format(part=sql.Identifier(f"{RROBIN_TABLE_PREFIX}{i}")))
        # Đánh số dòng và insert vào từng partition
        base_query = sql.SQL("""
            WITH numbered AS (
                SELECT userid, movieid, rating,
                       ROW_NUMBER() OVER () - 1 AS rn    -- bắt đầu từ 0
                FROM   {ratings}
            )
            """).format(ratings=sql.Identifier(ratingstablename))
        for i in range(numberofpartitions):
            cur.execute(base_query + sql.SQL("""
                INSERT INTO {part} (userid, movieid, rating)
                SELECT userid, movieid, rating
                FROM   numbered
                WHERE  rn % {n} = {i};
            """).format(
                part=sql.Identifier(f"{RROBIN_TABLE_PREFIX}{i}"),
                n=sql.Literal(numberofpartitions),
                i=sql.Literal(i)
            ))
        # Reset lại sequence và metadata
        cur.execute("""
            DROP SEQUENCE IF EXISTS rr_seq CASCADE;
            CREATE SEQUENCE rr_seq;
        """)
        cur.execute("""
            DROP TABLE IF EXISTS roundrobin_metadata;
            CREATE TABLE roundrobin_metadata (
                num_partitions INTEGER NOT NULL,
                CONSTRAINT rr_meta_pk CHECK (num_partitions > 0)
            );
        """)
        cur.execute("INSERT INTO roundrobin_metadata VALUES (%s);", (numberofpartitions,))
        openconnection.commit()
```

Hình 7 – Chi tiết hàm *roundrobinpartition()*

Mục đích: Tạo N phân mảnh vòng tròn của bảng ratings và lưu vào PostgreSQL. Phân chia các bản ghi trong bảng ratings vào các phân vùng tương ứng.

Các tham số:

- ratingstablename: là bảng ratings cần phân mảnh

- `numberofpartitions`: một số nguyên N là số phân vùng cần tạo
- `openconnection`: đối tượng kết nối đến cơ sở dữ liệu

Phân tích chi tiết:

- Khởi tạo một con trỏ để thực hiện truy vấn.
- Hằng số `RROBIN_TABLE_PREFIX` được sử dụng để làm tiền tố cho tên các bảng phân mảnh. Các bản phân mảnh sẽ có tên dạng `rrobin_part0`, `rrobin_part1`, ..., giúp phân biệt một cách dễ dàng.
- Tìm các bảng phân vùng cũ, duyệt qua từng bảng và sử dụng câu lệnh `DROP TABLE IF EXISTS` để xóa các bảng này nếu chúng tồn tại. Điều này đảm bảo xóa hết các phân mảnh cũ, tránh gây lỗi khi tạo phân mảnh mới.
- Tạo các bảng phân mảnh (`rrobin_part0`, `rrobin_part1`, ...). Các bảng là phân mảnh ngang của bảng `ratings` nên có cấu trúc gồm 3 cột `UserId`, `MovieId`, `Rating` giống bảng `ratings`.
- Phân phối dữ liệu vào các phân mảnh:
 - Sử dụng `ROW_NUMBER` để đánh số thứ tự các bản ghi trong bảng `rating` bắt đầu từ số 0.
 - Áp dụng công thức:

$$i = rn \% n$$
 Với i là chỉ số phân vùng, rn là số thứ tự của bản ghi và n là số phân vùng cần tạo.
 - Như vậy, bản ghi thứ 0, n , $2n$, ... sẽ được lưu vào phân vùng `rrobin_part0`; bản ghi thứ 1, $n+1$, $n+2$, ... sẽ được lưu vào phân vùng `rrobin_part1`, ...
- Tạo Sequence `rr_seq` để theo dõi vị trí chèn tiếp theo, đồng thời đảm bảo việc phân phối round robin chính xác ngay cả khi có nhiều insert đồng thời.
- Tạo bảng `roundrobin_metadata` gồm thuộc tính `num_partitions` để lưu số lượng phân vùng.
- Cuối cùng commit để lưu tất cả thay đổi đã thực hiện và đóng con trỏ.

3.2.2.5 Hàm *roundrobininsert()*

```
def roundrobininsert(ratingtablename, userid, itemid, rating, openconnection):
    """
    Thêm 1 dòng vào bảng rating và partition tương ứng theo round robin.
    """
    with openconnection.cursor() as cur:
        # Lấy số lượng partition
        cur.execute(sql.SQL("SELECT num_partitions FROM {m};").format(m=sql.Identifier(META_TABLE)))
        row = cur.fetchone()
        if row is None:
            raise RuntimeError("Thiếu thông tin metadata round robin")
        n_partitions = row[0]
        # Lấy slot tiếp theo (atomic)
        cur.execute(sql.SQL("SELECT nextval({seq});").format(seq=sql.Literal(SEQ_NAME)))
        slot = cur.fetchone()[0]
        target_index = (slot - 1) % n_partitions
        target_table = f"{RRROBIN_TABLE_PREFIX}{target_index}"
        # Insert vào bảng chính và bảng partition
        insert_sql = sql.SQL("INSERT INTO {tbl} (userid, movieid, rating) VALUES (%s, %s, %s)")
        for tbl in (ratingtablename, target_table):
            cur.execute(insert_sql.format(tbl=sql.Identifier(tbl)), (userid, itemid, rating))
        openconnection.commit()
```

Hình 8 – Chi tiết hàm *roundrobininsert()*

Mục đích: Chèn một bản ghi vào bảng ratings, sau đó lấy thông tin bảng phân vùng tương ứng để chèn bản ghi này vào đúng phân vùng.

Các tham số:

- ratingtablename: là bảng ratings trong PostgreSQL
- userid, itemid, rating: dữ liệu của một bản ghi
- openconnection: đối tượng kết nối đến cơ sở dữ liệu

Phân tích chi tiết:

- Lấy số lượng phân vùng từ bảng roundrobin_metadata.
- Lấy giá trị tiếp theo là một số nguyên từ Sequence rr_seq
- Tính chỉ số phân vùng bằng công thức:

$$target_index = (slot - 1) \% n_partitions$$

Với slot là giá trị lấy được từ rr_seq và n_partitions là số lượng phân vùng lấy từ roundrobin_meatadata.

- Chèn bản ghi vào bảng ratings và phân vùng đã tính được ở trên.
- Cuối cùng commit để lưu tất cả thay đổi đã thực hiện và đóng con trỏ.

CHƯƠNG 4. CÀI ĐẶT, KIỂM THỬ

4.1 Môi trường cài đặt

IDE: Pycharm 2024.3.5

Cơ sở dữ liệu: PostgreSQL

Phiên bản Python: Python 3.12

4.2 Kết quả triển khai

4.2.1 Đọc dữ liệu ratings

Các bước thực hiện:

1. Chạy file Assignment1Tester.py với dữ liệu là file ratings.dat.
2. Đọc dữ liệu bằng hàm loadratings().

Kết quả mong đợi:

- Vượt qua hết các test case.
- Trong cơ sở dữ liệu xuất hiện bảng ratings với đủ các bản ghi

Kết quả triển khai:

```
Created table ratings
Data inserted into ratings successfully
loadratings function pass!
```

Hình 9 – Kết quả triển khai hàm loadratings()

	userid integer	movieid integer	rating double precision
1	1	122	5
2	1	185	5
3	1	231	5
4	1	292	5
5	1	316	5
6	1	329	5
7	1	355	5
8	1	356	5
9	1	362	5
Total rows: 10000054		Query complete 00:00:02.761	

Hình 10 – Dữ liệu trong bảng ratings

Kết luận: Hàm loadratings() đã vượt qua hết các test case và trong cơ sở dữ liệu đã xuất hiện bảng ratings với đủ 10000054 bản ghi.

4.2.2 Phân mảnh theo khoảng

Các bước thực hiện:

1. Chạy file Assignment1Tester.py với dữ liệu là file ratings.dat.
2. Đọc dữ liệu bằng hàm loadratings().
3. Phân mảnh bảng ratings thành 5 bảng theo giá trị của cột Rating.

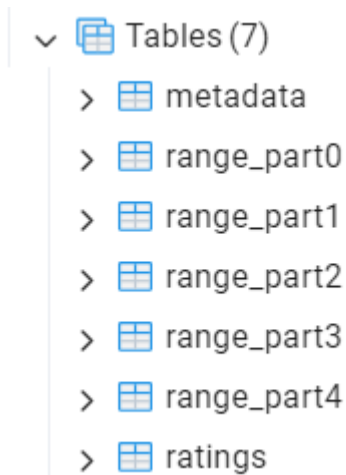
Kết quả mong đợi:

- Vượt qua hết các test case.
- Bảng ratings đã được phân mảnh thành 5 bảng range_part.
- Các bản ghi được chèn vào đúng các phân mảnh.

Kết quả triển khai:

```
Created table ratings
Data inserted into ratings successfully
loadratings function pass!
rangepartition function pass!
```

Hình 11 – Kết quả triển khai hàm rangepartition()



Hình 12 – Các bảng trong CSDL sau khi phân mảnh theo khoảng

	userid integer	movieid integer	rating double precision
1	4	231	1
2	5	1	1
3	5	708	1
4	5	736	1
5	5	780	1
6	5	1391	1
7	6	3986	1
8	6	4270	1
9	7	1917	1
Total rows: 479168		Query complete 00:00:00.185	

Hình 13 – Dữ liệu trong bảng range_part0

Kết luận: Hàm rangepartition() đã vượt qua hết các test case và bảng ratings đã được phân mảnh thành 5 bảng nhỏ theo giá trị của Ratings, kiểm tra cho thấy các bản ghi đã được chia về đúng phân mảnh.

4.2.3 Chèn dữ liệu vào phân mảnh theo khoảng

Các bước thực hiện:

1. Chạy file Assigment1Tester.py với dữ liệu là file ratings.dat.
2. Đọc dữ liệu bằng hàm loadratings().
3. Phân mảnh bảng ratings thành 5 bảng theo giá trị của cột Rating.

- Chèn một bản ghi userid = 100, itemid = 2, rating = 3 vào bảng

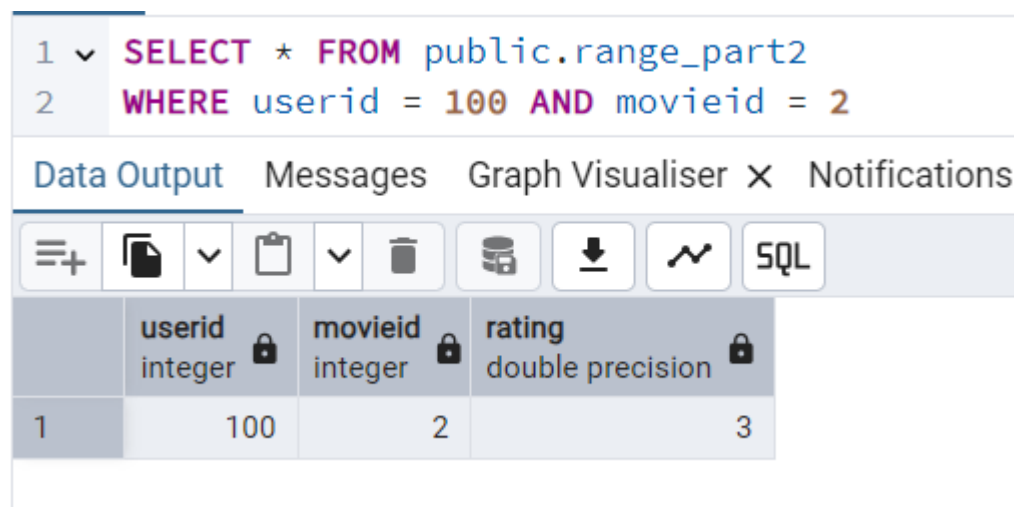
Kết quả mong đợi:

- Vượt qua hết các test case.
- Bản ghi được chèn vào đúng phân mảnh range_part2

Kết quả triển khai:

```
Created table ratings
Data inserted into ratings successfully
loadratings function pass!
rangepartition function pass!
rangeinsert function pass!
```

Hình 14 – Kết quả triển khai hàm rangeinsert()



The screenshot shows a SQL query execution interface. The query is: `SELECT * FROM public.range_part2 WHERE userid = 100 AND movieid = 2`. Below the query, there are tabs for 'Data Output', 'Messages', 'Graph Visualiser', and 'Notifications'. The 'Data Output' tab is selected, showing a table with the following data:

	userid integer	movieid integer	rating double precision
1	100	2	3

Hình 15 – Truy vấn kiểm tra dữ liệu trong bảng range_part2

Kết luận: Hàm rangeinsert() đã vượt qua hết các test case và bản ghi mới đã được chèn vào đúng phân mảnh.

4.2.4 Phân mảnh vòng tròn

Các bước thực hiện:

- Chạy file Assigment1 Tester.py với dữ liệu là file ratings.dat.
- Đọc dữ liệu bằng hàm loadratings().
- Phân mảnh bảng ratings thành 5 bảng theo thuật toán vòng tròn.

Kết quả mong đợi:

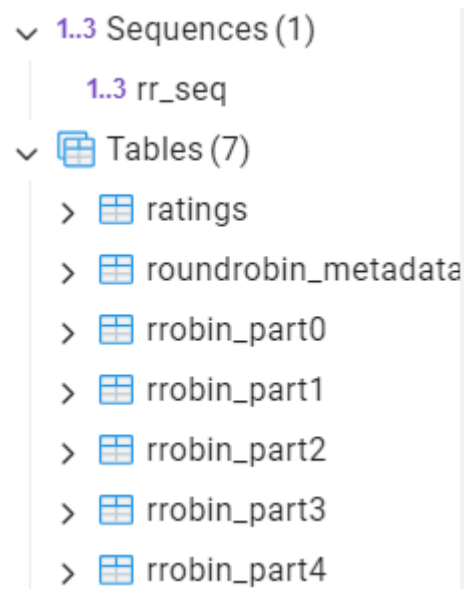
- Vượt qua hết các test case.

- Bảng ratings đã được phân mảnh thành 5 bảng rrobin_part.
- Các bản ghi được chèn vào đúng các phân mảnh.

Kết quả triển khai:

```
Created table ratings
Data inserted into ratings successfully
loadratings function pass!
roundrobinpartition function pass!
```

Hình 16 – Kết quả triển khai hàm roundrobinpartition()



Hình 17 – Các bảng trong CSDL sau khi phân mảnh vòng tròn

	userid integer	movieid integer	rating double precision
1	1	122	5
2	1	329	5
3	1	370	5
4	1	520	5
5	1	594	5
6	2	376	3
7	2	733	3
8	2	858	2
9	2	1391	3
Total rows: 2000011		Query complete 00:00:00.510	

Hình 18 – Dữ liệu trong bảng rrobin_part0

Kết luận: Hàm roundrobinpartition() đã vượt qua hết các test case và bảng ratings đã được phân mảnh thành 5 bảng nhỏ theo thuật toán vòng tròn, kiểm tra cho thấy các bản ghi đã được chia về đúng phân mảnh.

4.2.5 Chèn dữ liệu vào phân mảnh vòng tròn

Các bước thực hiện:

5. Chạy file Assignment1 Tester.py với dữ liệu là file ratings.dat.
6. Đọc dữ liệu bằng hàm loadratings().
7. Phân mảnh bảng ratings thành 5 bảng theo giá trị của cột Rating.
8. Chèn một bản ghi userid = 100, itemid = 1, rating = 3 vào bảng
9. Chèn một bản ghi userid = 101, itemid = 2, rating = 3 vào bảng

Kết quả mong đợi:

- Vượt qua hết các test case.
- Bản ghi đầu tiên được chèn vào đúng phân mảnh rrobin_part0.
- Bản ghi thứ hai được chèn vào đúng phân mảnh rrobin_part1.

Kết quả triển khai:

```
Created table ratings
Data inserted into ratings successfully
loadratings function pass!
roundrobinpartition function pass!
roundrobininsert function pass!
```

Hình 19 – Kết quả triển khai hàm roundrobininsert()

1

▼

SELECT * FROM public.rrobin_part0

2

WHERE

userid = 100 AND movieid = 1 AND rating = 3

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

	userid integer	movieid integer	rating double precision
1	100	1	3

Hình 20 – Truy vấn kiểm tra dữ liệu trong bảng rrobin_part0

1

▼

SELECT * FROM public.rrobin_part1

2

WHERE

userid = 101 AND movieid = 2 AND rating = 3

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

	userid integer	movieid integer	rating double precision
1	101	2	3

Hình 21 – Truy vấn kiểm tra dữ liệu trong bảng rrobin_part1

Kết luận: Hàm roundrobininsert() đã vượt qua hết các test case và các bản ghi mới đã được chèn vào đúng phân mảnh.

TÀI LIỆU THAM KHẢO

- [1] “What is Python – Python language explained.” *Amazon Web Service*,
<https://aws.amazon.com/what-is/python/>
- [2] “PostgreSQL là gì? Tìm hiểu về hệ quản trị cơ sở dữ liệu PostgreSQL.” *FPT Cloud*,
<https://fptcloud.com/postgresql-la-gi/>
- [3] “When and how to use Psycopg2.” *Timescale*,
<https://www.timescale.com/blog/when-and-how-to-use-psycopg2/>